Síntoma SELECT

Conversational Agent for Identifying and Analyzing Chronic Patient Cohorts using Natural Language and AI

Technical Documentation

Dedalus Datathon · Castilla-La Mancha 2025

Authors:	Hugo Cantero Serrano Diego Calcerrada Romero
Degree:	Computer Engineering
University:	Universidad de Castilla-La Mancha (UCLM)
Repository:	`github.com/kante420/cohort-agent`
Date:	April 2025

Abstract

Síntoma SELECT is a conversational agent designed to enable healthcare professionals to identify and analyze cohorts of chronic patients through natural language queries, without requiring any knowledge of SQL or programming.

The system translates plain-language questions into SQL queries using a locally-hosted large language model (LLM), executes them against a DuckDB analytical database loaded with six clinical cohort CSV files, and returns natural language responses accompanied by automatically generated visualizations.

The architecture is built around **LangGraph** as the orchestration framework, **Qwen 2.5 Coder 14B Q8** as the language model (served via Ollama on a remote GPU), **DuckDB** as the query engine, **Plotly** for visualization, and **Streamlit** for the user interface.

Key technical contributions include: a multi-node LangGraph graph with intent-based routing, a prompt engineering pipeline that minimizes hallucinations, a SQL correction layer (`fix_sql()`) with nine automated rules, session memory that includes previous SQL context for multi-turn conversations, and a modular architecture designed for scalability.

Keywords: conversational AI, natural language to SQL, LangGraph, LLM, DuckDB, chronic patients, cohort identification, prompt engineering, healthcare informatics.

Contents

Abstract **1**

1 Introduction **4**
- 1.1 Context and Motivation . . . 4
- 1.2 Challenge Statement . . . 4
- 1.3 Document Structure . . . 4

2 Data Sources and Database Schema **6**
- 2.1 Cohort Dataset . . . 6
  - 2.1.1 File Overview . . . 6
  - 2.1.2 Table Schemas . . . 6
- 2.2 Database Loading . . . 8

3 System Architecture **9**
- 3.1 Overview . . . 9
- 3.2 Project Structure . . . 9

3.3 The AgentState Dictionary . . . 10
3.4 Request Lifecycle . . . 10

4 Prompt Engineering **11**
4.1 Philosophy . . . 11
4.2 SQL Generation Prompt . . . 11
4.2.1 Schema Description . . . 11
4.2.2 Rule Set . . . 11
4.2.3 Few-Shot Examples . . . 12
4.3 Answer Generation Prompt . . . 12
4.4 The fix_sql() Post-Processing Layer . . . 13

5 LangGraph Graph and Node Implementations **15**
5.1 Graph Definition . . . 15
5.2 Intent Node . . . 16
5.3 SQL Node . . . 16
5.4 Viz Node . . . 16
5.5 Action Node . . . 16
5.6 Session Memory . . . 17

6 Visualization Subsystem **18**
6.1 Chart Type Detection . . . 18
6.2 Chart Types . . . 19
6.3 Visual Enhancements . . . 19

7 User Interface **20**
7.1 Chat Interface . . . 20
7.2 Sidebar . . . 20
7.3 Quick Actions . . . 20
7.4 PDF Export . . . 20

8 LLM and GPU Infrastructure **21**
8.1 Model Selection . . . 21
8.2 Ollama as the Inference Server . . . 21
8.3 Remote GPU Infrastructure . . . 22

9 Results and Evaluation **23**
9.1 Test Query Suite . . . 23
9.1.1 Simple Aggregations . . . 23
9.1.2 Condition-Based Filters . . . 23
9.1.3 Multi-Condition Intersection . . . 23
9.1.4 Temporal Filters . . . 23
9.1.5 Geographic Cross-Table Queries . . . 24
9.1.6 Multi-Turn Context . . . 24
9.2 Failure Cases and Resolutions . . . 24

10 Difficulties and Solutions **25**
10.1 LLM Hallucinations . . . 25
10.2 Model Size Limitations . . . 25
10.3 GPU Infrastructure Availability . . . 25
10.4 CSV Column Names with Special Characters . . . 25
10.5 StringDtype vs object in Pandas . . . 26

10.6 StreamlitDuplicateElementId 26

11 Conclusions and Future Work **27**
11.1 Conclusions 27
11.2 Future Work 27
11.2.1 Larger Datasets 27
11.2.2 Map Visualization 27
11.2.3 Dashboard with Key Metrics 28
11.2.4 Expanded Action Set 28
11.2.5 Authentication and Multi-User Support 28
11.2.6 Model Fine-Tuning 28

A Installation and Setup **29**
A.1 Prerequisites 29
A.2 Installation Steps 29

B Dependencies **30**

C Sample Queries and Expected Outputs **31**

Chapter 1

Introduction

1.1 Context and Motivation

Chronic disease management represents one of the most significant challenges in modern healthcare systems. Identifying specific patient cohorts — groups sharing one or more clinical characteristics — is essential for epidemiological analysis, treatment optimization, resource allocation, and clinical research.

However, the tools available for cohort identification typically require technical expertise. SQL queries, database management systems, and analytical platforms are inaccessible to most clinicians, nurses, and healthcare administrators who are the primary consumers of such information. This creates a bottleneck: the professionals who most need to answer clinical questions are the least able to interact directly with the data.

Síntoma SELECT addresses this gap by providing a conversational interface that accepts questions in natural Spanish and returns clinical insights, charts, and exportable data — without exposing any technical complexity to the end user.

1.2 Challenge Statement

This project was developed as a solution to the challenge proposed by **Dedalus** for their Datathon in Castilla-La Mancha (2025). The challenge required building a conversational agent for chronic patient cohort identification meeting the following criteria:

- **Minimize hallucinations**: Answers must be grounded in real data and not invented by the model.
- **Cohort statistics**: The system must provide statistical summaries, distributions, and comparisons.
- **Non-textual rendering**: Results should include charts and visual representations when appropriate.
- **Actions on cohorts**: Users must be able to export and act on query results.
- **Modular architecture**: The solution must be scalable and maintainable.
- **Prompt engineering**: Techniques must be employed to guarantee data quality and response reliability.

1.3 Document Structure

This document is organized as follows:

- **Chapter 2** describes the data sources and database schema.

- **Chapter 3** presents the system architecture and design decisions.
- **Chapter 4** details the prompt engineering techniques applied.
- **Chapter 5** documents the LangGraph graph and node implementations.
- **Chapter 6** covers the visualization subsystem.
- **Chapter 7** describes the Streamlit user interface.
- **Chapter 8** discusses the LLM selection and GPU infrastructure.
- **Chapter 9** presents results, test cases, and evaluation.
- **Chapter 10** documents the difficulties encountered and solutions applied.
- **Chapter 11** concludes with future work directions.

Chapter 2

Data Sources and Database Schema

2.1 Cohort Dataset

The system operates over six CSV files representing a synthetic cohort of chronic patients. These files are loaded into a **DuckDB** in-process analytical database at startup and queried in real time during conversations.

2.1.1 File Overview

File	Description	Records
`cohorte_pacientes.csv`	Patient demographics	33
`cohorte_condiciones.csv`	Clinical conditions (SNOMED CT)	100
`cohorte_encuentros.csv`	Medical encounters	112
`cohorte_medicaciones.csv`	Prescribed medications	100
`cohorte_alergias.csv`	Allergies (SNOMED CT)	100
`cohorte_procedimientos.csv`	Clinical procedures	100

Table 2.1: Cohort dataset files

2.1.2 Table Schemas

cohorte_pacientes

Column	Type	Description
PacienteID	BIGINT	Unique patient identifier (primary key)
Genero	VARCHAR	Gender (`Masculino` / `Femenino`)
Edad	BIGINT	Age in years
Provincia	VARCHAR	Province of residence
Latitud	DOUBLE	Geographic latitude
Longitud	DOUBLE	Geographic longitude

Table 2.2: Schema: cohorte_pacientes

cohorte_condiciones

Column	Type	Description
PacienteID	BIGINT	Foreign key to cohorte_pacientes
Fecha_inicio	TIMESTAMP	Condition onset date
Fecha_fin	TIMESTAMP	Condition end date (NULL if active)
Codigo_SNOMED	VARCHAR	SNOMED CT code
Descripcion	VARCHAR	Human-readable condition name

Table 2.3: Schema: cohorte_condiciones

cohorte_encuentros

Column	Type	Description
PacienteID	BIGINT	Foreign key to cohorte_pacientes
Tipo_encuentro	VARCHAR	Type of medical visit
Fecha_inicio	DATE	Encounter start date
Fecha_fin	DATE	Encounter end date

Table 2.4: Schema: cohorte_encuentros — no auto-increment ID column

cohorte_medicaciones

Column	Type	Description
PacienteID	BIGINT	Foreign key to cohorte_pacientes
Fecha_inicio	DATE	Prescription start date
Fecha_fin	DATE	End date (NULL if active)
Codigo	VARCHAR	Medication code
Nombre	VARCHAR	Medication name
Dosis	VARCHAR	Prescribed dose
Frecuencia	VARCHAR	Administration frequency
Vía_administración	VARCHAR	Administration route

Table 2.5: Schema: cohorte_medicaciones

cohorte_alergias

Column	Type	Description
PacienteID	BIGINT	Foreign key to cohorte_pacientes
Fecha_diagnostico	DATE	Allergy diagnosis date
Codigo_SNOMED	BIGINT	SNOMED CT code (integer type)
Descripcion	VARCHAR	Allergy description

Table 2.6: Schema: cohorte_alergias

cohorte_procedimientos

Column	Type	Description
PacienteID	BIGINT	Foreign key to cohorte_pacientes
Fecha_inicio	DATE	Procedure start date
Fecha_fin	DATE	Procedure end date
Codigo_SNOMED	BIGINT	SNOMED CT code (integer type)
Descripcion	VARCHAR	Procedure description

Table 2.7: Schema: cohorte_procedimientos

2.2 Database Loading

The `db/setup.py` module loads all CSV files into DuckDB at startup using `read_csv_auto`:

```
def load_csvs_to_duckdb():
    con = duckdb.connect(DB_PATH)
    for key, table_name in TABLE_NAMES.items():
        csv_path = os.path.join(CSV_DIR, f"{table_name}.csv")
        con.execute(f"""
            CREATE OR REPLACE TABLE {table_name} AS
            SELECT * FROM read_csv_auto('{csv_path}', header=
               True)
        """)
    con.close()
```

Listing 2.1: CSV loading into DuckDB

All subsequent queries use `read_only=True` connections to prevent any data modification.

Chapter 3

System Architecture

3.1 Overview

The system is organized into five distinct layers, each with a clearly defined responsibility:

1. **Presentation layer**: Streamlit chat interface with sidebar actions.
2. **Orchestration layer**: LangGraph graph with intent routing and session memory.
3. **Language model layer**: Qwen 2.5 Coder 14B Q8 via Ollama (local or remote GPU).
4. **Data layer**: DuckDB query engine over six CSV cohort files.
5. **Visualization layer**: Plotly chart generation based on query result shape.

3.2 Project Structure

```
cohort_agent/
          app.py                    # Streamlit UI entry point
          config.py                 # Environment variables
          data/raw/                 # Cohort CSV files
          db/
                setup.py              # CSV -> DuckDB loader
          agent/
                graph.py              # LangGraph graph
   definition
                memory.py             # Session memory (
   ConversationMemory)
                nodes/
                    intent.py         # Intent detection node
                    sql_node.py       # SQL generation and
   execution
                    viz_node.py       # Chart generation node
                    action_node.py    # Export and quick action
   node
          prompts/
                sql_prompt.py         # Prompt engineering
   templates
          viz/
              charts.py             # Plotly chart builder
              export.py             # PDF conversation export
```

Listing 3.1: Project directory structure

3.3 The AgentState Dictionary

All LangGraph nodes communicate through a shared `AgentState` dictionary. Each node reads the keys it needs and writes the keys it produces, without modifying or deleting prior entries:

```
class AgentState(TypedDict, total=False):
    user_question:          str    # Original user message
    conversation_history:   str    # Serialized session memory
    intent:                 str    # Classified intent
    sql:                    str    # Generated SQL query
    query_result:           Optional[object]  # pandas
        DataFrame
    answer:                 str    # Natural language response
    chart:                  Optional[object]  # Plotly Figure
    error:                  Optional[str]     # Error message
        if any
```

Listing 3.2: AgentState TypedDict definition

This design means adding a new node only requires defining which keys it reads and writes — no changes needed to existing nodes.

3.4 Request Lifecycle

A complete request follows this sequence:

1. User submits a message via Streamlit chat input.
2. `app.py` constructs the initial state with `user_question` and `conversation_history`.
3. LangGraph invokes the graph starting at `intent_node`.
4. `intent_node` classifies the message as `sql`, `estadistica`, `accion`, or `fuera_scope`.
5. The router directs the state to the appropriate node.
6. For SQL/statistics: `sql_node` generates SQL, validates it, executes it via DuckDB, and generates a natural language answer.
7. `viz_node` examines the DataFrame and generates a Plotly chart if appropriate.
8. The state arrives at `END` with `answer` and optionally `chart` and `sql`.
9. `app.py` renders the response, chart, and SQL expander.
10. `ConversationMemory.add_turn()` stores the message, answer, and SQL for future context.

Chapter 4

Prompt Engineering

4.1 Philosophy

Prompt engineering is the primary mechanism for hallucination prevention in this system. Rather than relying on the model's general knowledge, every prompt is constructed to provide exactly the information the model needs and to constrain its output format and content precisely.

Two prompts are used: `SQL_GENERATION_PROMPT` (translates the question to SQL) and `ANSWER_GENERATION_PROMPT` (translates the query result to natural language).

4.2 SQL Generation Prompt

4.2.1 Schema Description

The first component is a complete, exact description of the database schema. This is the most critical anti-hallucination measure: the model cannot invent column names or table names because the correct ones are provided explicitly.

> **Key principle:** The LLM never "knows" the database. It reads the schema description in the prompt on every request, like a developer consulting documentation before writing a query.

The schema description includes table names, column names, data types, and semantic descriptions. For tables with type-sensitive columns (e.g., `Codigo_SNOMED` as `BIGINT` in procedures and allergies), explicit notes are included.

4.2.2 Rule Set

Fourteen strict rules govern SQL generation:

Rule 1. Return SQL only — no explanations, markdown, or ```sql fences.

Rule 2. Use only tables and columns from the schema description.

Rule 3. Always use `ILIKE` with `%` for text searches.

Rule 4. Apply `LIMIT 100` unless otherwise requested.

Rule 5. Use the most conservative query when the question is ambiguous.

Rule 6. `Fecha_fin IS NULL` only in `cohorte_condiciones` and `cohorte_medicaciones` (active records). Never in `cohorte_procedimientos` or `cohorte_encuentros`.

Rule 7. Never use `DROP`, `DELETE`, `UPDATE`, `INSERT`, or any write operation.

Rule 8. `CANNOT_ANSWER` only if the question cannot be answered with these six tables.

Rule 9. `ILIKE` and `LIKE` only on `VARCHAR` columns. Never on `Codigo_SNOMED` (BIGINT).

Rule 10. Never use `Codigo_SNOMED` to search by disease or procedure name. Always use `Descripcion` with `ILIKE`.

Rule 11. Maintain alias consistency: if `cohorte_condiciones AS cc`, use `cc.columna` throughout.

Rule 12. `cohorte_encuentros` has no auto-increment ID — use `COUNT(*)` to count encounters.

Rule 13. For gender values: map "hombre" / "varón" to `'Masculino'`, "mujer" / "femenina" to `'Femenino'`.

Rule 14. Use conversation history to understand follow-up questions and reuse previous `WHERE` clauses.

4.2.3 Few-Shot Examples

Four critical query patterns are demonstrated via few-shot examples:

```sql
-- Question: What is the average age of patients with diabetes
   ?
SELECT AVG(cp.Edad) AS Edad_Media
FROM cohorte_pacientes cp
JOIN cohorte_condiciones cc ON cp.PacienteID = cc.PacienteID
WHERE cc.Descripcion ILIKE '%diabetes%'
```

Listing 4.1: Few-shot: average age with condition filter

```sql
-- Question: How many patients have diabetes and hypertension?
SELECT COUNT(DISTINCT cp.PacienteID) AS Total
FROM cohorte_pacientes cp
JOIN cohorte_condiciones cc1 ON cp.PacienteID = cc1.PacienteID
JOIN cohorte_condiciones cc2 ON cp.PacienteID = cc2.PacienteID
WHERE cc1.Descripcion ILIKE '%diabetes%'
  AND cc2.Descripcion ILIKE '%hipertension%'
```

Listing 4.2: Few-shot: patients with two simultaneous conditions

These examples teach the model the double-JOIN pattern for multi-condition queries — a pattern that small models fail to infer without explicit demonstration.

4.3 Answer Generation Prompt

The answer prompt receives the original question, the executed SQL, the result row count, and a text representation of the DataFrame (up to 20 rows). It instructs the model to:

- Respond in Spanish.
- Never mention SQL or technical terminology.
- Highlight relevant numerical values.
- Keep the response to a maximum of four sentences.

- List ALL values from the result — never omit any.

4.4 The fix_sql() Post-Processing Layer

Even with careful prompt engineering, small models generate syntactically or semantically incorrect SQL at some frequency. The `fix_sql()` function acts as a second layer of correction, applying nine automated rules via regular expressions before the query reaches DuckDB:

```python
def fix_sql(sql: str) -> str:
    import re

    # Rule 1: ILIKE/LIKE on Codigo_SNOMED -> use Descripcion
    sql = re.sub(r'Codigo_SNOMED\s+(I?LIKE)\s+',
                 r'Descripcion \1 ', sql, flags=re.IGNORECASE)

    # Rule 2: Codigo_SNOMED = <number> -> SNOMED hallucination
    if re.search(r'Codigo_SNOMED\s*=\s*[\'"]?\d+[\'"]?',
                 sql, flags=re.IGNORECASE):
        return "SNOMED_HALLUCINATION"

    # Rule 3: Remove Fecha_fin IS NULL from procedures/encounters
    for tabla in ["cohorte_procedimientos", "cohorte_encuentros"]:
        if tabla.lower() in sql.lower():
            sql = re.sub(r'AND\s+Fecha_fin\s+IS\s+NULL', '',
                         sql, flags=re.IGNORECASE)

    # Rule 4: Fix inconsistent aliases (T1, T2, c, p -> correct alias)
    alias_map = {}
    for match in re.finditer(r'\b\w+\s+AS\s+(\w+)\b',
                             sql, flags=re.IGNORECASE):
        alias_map[match.group(1).upper()] = match.group(1)

    used_prefixes = re.findall(r'\b([A-Za-z_]\w*)\.', sql)
    for prefix in set(used_prefixes):
        if prefix.upper() not in alias_map:
            candidates = [a for a in alias_map
                          if a.upper().startswith(prefix[0].upper())]
            if len(candidates) == 1:
                sql = re.sub(rf'\b{re.escape(prefix)}\.',
                             f'{alias_map[candidates[0]]}.',
                             sql)

    # Rule 5: Add DISTINCT to JOINs lacking it
    if re.search(r'JOIN', sql, flags=re.IGNORECASE):
        sql = re.sub(r'\bSELECT\b(?!\s+DISTINCT)',
                     'SELECT DISTINCT', sql, flags=re.IGNORECASE)
```

```python

    # Rule 6: LIKE -> ILIKE (DuckDB is case-sensitive with LIKE)
    sql = re.sub(r'\bLIKE\b(?!\s+ALL)(?!\s+ANY)',
                 'ILIKE', sql, flags=re.IGNORECASE)

    # Rule 7: YEAR() -> EXTRACT(YEAR FROM ...)
    sql = re.sub(r'\bYEAR\s*\(\s*(\w+\.\w+|\w+)\s*\)',
                 r'EXTRACT(YEAR FROM \1)', sql, flags=re.IGNORECASE)

    # Rule 8: MONTH() -> EXTRACT(MONTH FROM ...)
    sql = re.sub(r'\bMONTH\s*\(\s*(\w+\.\w+|\w+)\s*\)',
                 r'EXTRACT(MONTH FROM \1)', sql, flags=re.IGNORECASE)

    # Rule 9: EncuentroID does not exist -> COUNT(*)
    sql = re.sub(r'COUNT\s*\(\s*DISTINCT\s+\w+\.EncuentroID\s*\)',
                 'COUNT(*)', sql, flags=re.IGNORECASE)

    return re.sub(r'\s+', ' ', sql).strip()
```

Listing 4.3: fix_sql() correction rules

Chapter 5

LangGraph Graph and Node Implementations

5.1 Graph Definition

The LangGraph graph is compiled once at module load time and reused across all requests. It defines five nodes and uses conditional edges for routing:

```python
def build_graph() -> StateGraph:
    graph = StateGraph(AgentState)

    graph.add_node("intent",      intent_node)
    graph.add_node("sql",         sql_node)
    graph.add_node("estadistica", estadistica_node)  # -> sql_node
    graph.add_node("accion",      action_node)
    graph.add_node("fuera_scope", fuera_scope_node)
    graph.add_node("viz",         viz_node)

    graph.set_entry_point("intent")

    graph.add_conditional_edges(
        "intent", route,
        {
            "sql":         "sql",
            "estadistica": "estadistica",
            "accion":      "accion",
            "fuera_scope": "fuera_scope",
        }
    )

    graph.add_edge("sql",         "viz")
    graph.add_edge("estadistica", "viz")
    graph.add_edge("viz",         END)
    graph.add_edge("accion",      END)
    graph.add_edge("fuera_scope", END)

    return graph.compile()
```

Listing 5.1: LangGraph graph construction

5.2 Intent Node

The intent node classifies each message into one of four categories using a compact prompt with ten few-shot examples:

Intent	Triggers
`sql`	Questions about specific patients, counts, lists, filters
`estadistica`	Summaries, averages, distributions, comparisons, charts
`accion`	Export to CSV, save results, download data
`fuera_scope`	Greetings, general questions, out-of-domain requests

Table 5.1: Intent classification categories

A defensive fallback maps any unrecognized output to `sql`, preventing routing failures.

5.3 SQL Node

The SQL node is the most complex component. It executes five steps sequentially:

1. **Generate SQL**: calls the LLM with the full `SQL_GENERATION_PROMPT`.
2. **Check for CANNOT_ANSWER**: exits early with a safe response if the LLM indicates the question cannot be answered.
3. **Clean markdown**: strips ‘‘‘`sql` fences that some models add despite instructions.
4. **Apply fix_sql()**: corrects known error patterns before validation.
5. **Validate**: runs `EXPLAIN` on the query to detect syntax errors before execution.
6. **Execute**: runs the validated query via DuckDB.
7. **Generate answer**: calls the LLM with `ANSWER_GENERATION_PROMPT` to produce a natural language response.

5.4 Viz Node

The viz node examines the DataFrame returned by the SQL node and calls `build_chart()` from `viz/charts.py`. It adds the resulting Plotly Figure (or `None`) to the state under the `chart` key.

5.5 Action Node

The action node handles requests to export data. If a previous query result exists in `state["query_result"]`, it is saved to `./exports/cohorte_exportada.csv`. If no prior data exists, it returns an informative message.

5.6 Session Memory

The `ConversationMemory` class uses a `collections.deque` with a configurable maximum of 10 turns (20 messages). Each turn stores the user message, the assistant response, and the SQL query executed:

```python
class ConversationMemory:
    def __init__(self, max_turns: int = 10):
        self._history = deque(maxlen=max_turns * 2)

    def add_turn(self, user_message: str,
                 assistant_response: str, sql: str = ""):
        self._history.append({"role": "user",
                              "content": user_message})
        self._history.append({"role": "assistant",
                              "content": assistant_response,
                              "sql": sql})

    def get_history_as_str(self) -> str:
        lines = []
        for msg in self._history:
            prefix = "Usuario" if msg["role"] == "user" \
                     else "Asistente"
            lines.append(f"{prefix}: {msg['content']}")
            if msg.get("sql"):
                lines.append(
                    f"[SQL ejecutado: {msg['sql']}]")
        return "\n".join(lines)
```

Listing 5.2: ConversationMemory implementation

Including the previous SQL in the history enables the model to handle follow-up questions like "what is their average age?" correctly — it can see the preceding `WHERE` clause and apply the same filter.

Chapter 6

Visualization Subsystem

6.1 Chart Type Detection

The `detect_chart_type()` function determines the most appropriate chart type by examining the DataFrame columns and the question text:

```
def detect_chart_type(df, question=""):
    text_cols = [c for c in cols
                 if pd.api.types.is_string_dtype(df[c])
                 or pd.api.types.is_object_dtype(df[c])]
    num_cols  = [c for c in cols
                 if pd.api.types.is_numeric_dtype(df[c])]
    date_cols = [c for c in cols
                 if any(k in c.lower()
                        for k in ["fecha","date","anio","year"
                           ])]

    wants_pie  = any(k in question.lower() for k in
                     ["porcentaje", "%", "quesito", "tarta",
                      "pie", "distribucion", "proporcion"])
    wants_hist = any(k in question.lower() for k in
                     ["distribucion de edad", "histograma"])
    wants_line = any(k in question.lower() for k in
                     ["por anio", "por mes", "evolucion",
                      "tendencia"])

    if date_cols and num_cols: return "line"
    if wants_line and num_cols: return "line"
    if wants_hist and num_cols: return "histogram"
    if n_cols == 1 and num_cols: return "histogram"
    if text_cols and num_cols:
        if wants_pie and df[text_cols[0]].nunique() <= 6:
            return "pie"
        if df[text_cols[0]].nunique() == 2 and wants_pie:
            return "pie"
        return "bar"
    return "table"
```

Listing 6.1: Chart type detection logic

6.2 Chart Types

Chart type	When generated
Bar chart	1 text column + 1 numeric column, more than 2 categories
Pie chart	Question contains percentage/distribution keywords, ≤6 categories
Area chart	Column name contains year/date keywords, or question mentions temporal trend
Histogram	Single numeric column, or question asks about age distribution
Table	Multiple text columns, no chart applies
None	Single-row single-column result (scalar answer)

Table 6.1: Chart type selection rules

6.3 Visual Enhancements

Bar charts use color-encoded bars (darker = higher value) with values displayed above each bar. Area charts display a filled area below the line with automatic Y-axis tick spacing. Pie charts use a slight `pull` effect on each sector. All charts use a clean white background with minimal grid lines.

Chapter 7

User Interface

7.1 Chat Interface

The main panel is a standard Streamlit chat interface using `st.chat_message` and `st.chat_input`. Each assistant response includes:

- The natural language answer rendered as markdown.
- A Plotly chart (if generated), rendered with `st.plotly_chart`.
- A collapsible "SQL ejecutado" expander showing the exact query.

Unique `key` parameters on each `st.plotly_chart` call prevent the `StreamlitDuplicateElementId` error that arises when re-rendering the message history.

7.2 Sidebar

The sidebar contains four sections:

1. **Actions**: Download last result as CSV, download full conversation as PDF, clear conversation.
2. **Quick Actions**: Direct patient lookup by ID (medication or full medical history), bypassing the LLM SQL generation entirely for maximum speed.
3. **About**: Project description and technology stack.
4. **Data files**: List of the six CSV files powering the agent.

7.3 Quick Actions

The Quick Actions expander provides a radio button to select the action type and a text input for the patient ID. On clicking "Consultar", a pre-defined SQL query is executed directly against DuckDB — the LLM is only used for the natural language answer generation, not for SQL generation. This reduces latency significantly for these common lookups.

7.4 PDF Export

The `conversation_to_pdf()` function in `viz/export.py` uses `reportlab` to generate a structured PDF including the conversation header, all user and assistant messages, and the SQL queries executed. The welcome message is excluded from exports via an `is_welcome` flag.

Chapter 8

LLM and GPU Infrastructure

8.1 Model Selection

The project iterated through three model sizes during development:

Model	Parameters	Quantization	Assessment
Qwen 2.5 Coder	3B	Q4	Frequent alias errors, SNOMED hallucinations, required many few-shot examples
Qwen 2.5 Coder	7B	Q4	Significant improvement, occasional alias inconsistencies
Qwen 2.5 Coder	14B	Q8	Resolves complex JOINs without examples, very few errors

Table 8.1: Model evaluation across development iterations

The **Qwen 2.5 Coder** family was chosen over general-purpose models because of its specific training on code generation tasks, which transfers well to SQL generation. The instruct variant (`qwen2.5-coder:14b-instruct-q8_0`) follows instructions more reliably than the base variant.

8.2 Ollama as the Inference Server

Ollama provides a unified REST API for local model serving. The Python `ollama` client library communicates with it via:

```python
import ollama
from config import OLLAMA_HOST

client = ollama.Client(host=OLLAMA_HOST)

response = client.chat(
    model="qwen2.5-coder:14b-instruct-q8_0",
    messages=[{"role": "user", "content": prompt}]
)
answer = response["message"]["content"].strip()
```

Listing 8.1: Ollama client usage

Setting `OLLAMA_HOST` in `.env` allows switching between local and remote inference with no code changes.

8.3 Remote GPU Infrastructure

During development, local inference on a 7B model was used. For the datathon demo, the model was deployed on a **Google Cloud VM** with an **NVIDIA L4 GPU** (24 GB VRAM) in region `us-east1-c`.

Specification	Value
GPU	NVIDIA L4
VRAM	24 GB
Machine type	g2-standard-4
vCPUs	4
RAM	16 GB
OS	Ubuntu 22.04 LTS
Ollama version	0.20.5
CUDA version	13.0
Driver version	580.126.09

Table 8.2: Google Cloud VM specifications

Ollama was configured to listen on all interfaces (`0.0.0.0:11434`) and a VPC firewall rule was created to allow TCP traffic on port 11434 from any source.

The **14B Q8** model requires approximately 15 GB of VRAM. The L4 (24 GB) provides sufficient headroom for the model plus the context window of each conversation turn.

> **Response times observed**: First request (model cold load): 15–20 seconds. Subsequent requests: 3–8 seconds. This performance is well within acceptable limits for a clinical query tool.

Chapter 9

Results and Evaluation

9.1 Test Query Suite

The following query categories were systematically tested during development:

9.1.1 Simple Aggregations

```sql
-- "How many patients are there per province?"
SELECT Provincia, COUNT(PacienteID) AS Total
FROM cohorte_pacientes
GROUP BY Provincia
ORDER BY Total DESC
```

Result: Correct, generates bar chart automatically.

9.1.2 Condition-Based Filters

```sql
-- "What patients have diabetes?"
SELECT DISTINCT cp.PacienteID
FROM cohorte_pacientes cp
JOIN cohorte_condiciones cc ON cp.PacienteID = cc.PacienteID
WHERE cc.Descripcion ILIKE '%diabetes%'
LIMIT 100
```

Result: Correct JOIN, no visualization (ID-only result).

9.1.3 Multi-Condition Intersection

```sql
-- "How many patients have both diabetes and hypertension?"
SELECT COUNT(DISTINCT cp.PacienteID) AS Total
FROM cohorte_pacientes cp
JOIN cohorte_condiciones cc1
  ON cp.PacienteID = cc1.PacienteID
JOIN cohorte_condiciones cc2
  ON cp.PacienteID = cc2.PacienteID
WHERE cc1.Descripcion ILIKE '%diabetes%'
  AND cc2.Descripcion ILIKE '%hipertension%'
```

Result: Correct double JOIN, returns 4. *This pattern fails entirely with the 3B model without few-shot examples.*

9.1.4 Temporal Filters

```sql
-- "What procedures were performed in 2024?"
SELECT DISTINCT p.Descripcion
FROM cohorte_procedimientos p
WHERE EXTRACT(YEAR FROM p.Fecha_inicio) = 2024
```

```
LIMIT 100
```

Result: Correct use of `EXTRACT` (after `fix_sql` Rule 7 converts `YEAR()`). Returns 10 distinct procedures.

9.1.5 Geographic Cross-Table Queries

```
-- "What conditions do patients from Granada have?"
SELECT DISTINCT cc.Descripcion
FROM cohorte_pacientes cp
JOIN cohorte_condiciones cc
  ON cp.PacienteID = cc.PacienteID
WHERE cp.Provincia ILIKE '%Granada%'
LIMIT 100
```

Result: Correct, returns 6 conditions, generates table visualization.

9.1.6 Multi-Turn Context

1. Query 1: "What medications do patients with diabetes take?"
2. Query 2: "What is their average age?"

Query 2 correctly reuses the diabetes filter from Query 1's SQL, which is included in the conversation history. Returns 46.5 years.

9.2 Failure Cases and Resolutions

Failure type	Example	Resolution
SNOMED hallucination	`Codigo_SNOMED = 840539007` for appendicitis	Rule 2 in fix_sql detects and blocks
Alias inconsistency	Defined `cc` but used `T2.columna`	Rule 4 in fix_sql corrects aliases
Wrong ILIKE target	`Codigo_SNOMED ILIKE '%diabetes%'`	Rule 1 in fix_sql redirects to `Descripcion`
Incorrect date filter	`Fecha_fin IS NULL` in procedures	Rule 3 in fix_sql removes it
Wrong function	`YEAR(Fecha_inicio)`	Rule 7 converts to `EXTRACT`

Table 9.1: Observed failure types and automated corrections

Chapter 10

Difficulties and Solutions

10.1 LLM Hallucinations

Problem: The language model frequently invented column names (e.g., `PatientID` instead of `PacienteID`), fabricated SNOMED codes, and applied text-search operators to integer columns.

Solution: A three-layer approach:

1. Strict schema description in the prompt with type annotations and notes.
2. Fourteen explicit rules covering the most common error patterns.
3. The `fix_sql()` post-processing function with nine automated corrections.

10.2 Model Size Limitations

Problem: The 3B model required large numbers of few-shot examples for basic JOIN patterns and still produced inconsistent results. Adding examples created a positive feedback loop — each new failure required another example.

Solution: Upgrading to progressively larger models (7B, then 14B). The 14B model resolves complex JOINs, double-table intersections, and temporal queries without any task-specific few-shot examples. This is a more principled solution than accumulating hundreds of examples.

10.3 GPU Infrastructure Availability

Problem: Three infrastructure failures occurred during development:

- Anthropic API key had insufficient credits.
- Gemini API free tier quota was zero despite account creation.
- vast.ai instance remained in "scheduling" state for over two hours.
- Google Cloud GPU quota was initially set to zero for new accounts.

Solution: Developed on local Ollama with the 7B model throughout. For the demo, manually requested GPU quota increase from Google Cloud support (approved within minutes), and deployed to a verified us-east1-c zone.

10.4 CSV Column Names with Special Characters

Problem: The `cohorte_medicaciones.csv` file contained column names with spaces (`Fecha de inicio`), accented characters (`Vía de administración`), and special characters. DuckDB requires double-quoted identifiers for such columns, but the model

consistently generated unquoted references, causing constant syntax errors.

Solution: Renamed all columns in the CSV file to use underscores and remove accents (`Fecha_inicio`, `Via_administracion`, etc.), then updated the schema description accordingly. This was significantly cleaner than maintaining a regex-based correction layer for all possible column name variants.

10.5 StringDtype vs object in Pandas

Problem: DuckDB returns string columns as pandas `StringDtype` rather than `object`. The chart detection code checked `df[c].dtype == object`, which returned `False` for `StringDtype` columns, causing all string columns to be invisible to the chart type detector and defaulting all categorical charts to table visualization.

Solution: Replaced the dtype check with `pd.api.types.is_string_dtype(df[c]) or pd.api.types.is_object_dtype(df[c])` throughout both `detect_chart_type()` and `build_chart()`.

10.6 StreamlitDuplicateElementId

Problem: Re-rendering the message history on each Streamlit rerun caused multiple `st.plotly_chart` calls with auto-generated identical IDs, resulting in a `StreamlitDuplicateElement` exception.

Solution: Added explicit `key=f"chart_history_i"` parameters to all `st.plotly_chart` calls, using the message index as a unique identifier.

Chapter 11

Conclusions and Future Work

11.1 Conclusions

Síntoma SELECT successfully demonstrates that a conversational agent can provide reliable, accurate clinical data access to non-technical users. The key findings from this project are:

1. **Prompt engineering is the primary anti-hallucination mechanism.** Providing the exact database schema, explicit rules, and strategic few-shot examples reduces model errors by an order of magnitude compared to relying on general model knowledge.
2. **Model size matters for complex SQL.** The 14B model handles multi-table JOINs, double-condition intersections, and temporal filters without task-specific examples. This is qualitatively different from the 3B and 7B models, which require extensive few-shot coaching.
3. **A post-processing correction layer is necessary.** Even well-prompted large models make systematic errors on specific patterns. A rule-based correction function provides a deterministic safety net that complements probabilistic prompt engineering.
4. **Session memory with SQL context enables meaningful multi-turn conversations.** Including the previous SQL in the conversation history allows the model to correctly handle follow-up questions without the user repeating filters.
5. **LangGraph enables clean modular architecture.** The intent-based routing graph allows each node to be developed, tested, and modified independently, making the system genuinely extensible.

11.2 Future Work

Several directions for improvement and extension are identified:

11.2.1 Larger Datasets

The current cohort is synthetic and contains only 33 patients. The architecture is designed to scale to real clinical datasets with millions of records — DuckDB handles analytical queries on large datasets efficiently, and the prompt engineering approach is dataset-size-independent.

11.2.2 Map Visualization

The `cohorte_pacientes` table includes `Latitud` and `Longitud` columns. A geographic choropleth or scatter map showing patient distribution across provinces would add significant visual impact and clinical utility.

11.2.3 Dashboard with Key Metrics

A fixed summary panel above the chat showing total patients, most frequent condition, average age, and most common procedure would provide immediate context and reduce the need for basic orientation queries.

11.2.4 Expanded Action Set

The current action node only supports CSV export. Future actions could include cohort filtering (saving a subset of patients for further analysis), report generation (structured clinical summaries), and comparison (two cohorts side by side).

11.2.5 Authentication and Multi-User Support

For clinical deployment, user authentication, role-based access control, and audit logging would be required. LangGraph's state management makes it straightforward to add user context to each request.

11.2.6 Model Fine-Tuning

With sufficient query-SQL pairs collected from real clinical usage, fine-tuning a smaller model on domain-specific data could yield better performance at lower computational cost than a general-purpose 14B model.

Appendix A

Installation and Setup

A.1 Prerequisites

- Python 3.12 or higher
- Ollama (from https://ollama.com/download)
- Git

A.2 Installation Steps

```
# 1. Clone the repository
git clone https://github.com/kante420/cohort-agent.git
cd cohort-agent

# 2. Create virtual environment
python -m venv .venv
.venv\Scripts\activate        # Windows
# source .venv/bin/activate   # macOS/Linux

# 3. Install dependencies
pip install -r requirements.txt

# 4. Pull the LLM model
ollama pull qwen2.5-coder:14b-instruct-q8_0

# 5. Create .env file
echo DB_PATH=./db/cohort.duckdb > .env
echo OLLAMA_HOST=http://localhost:11434 >> .env

# 6. Copy CSV files to data/raw/ and load database
python -m db.setup

# 7. Launch the application
streamlit run app.py
```

Appendix B

Dependencies

Package	Version	Purpose
langgraph	≥0.2.0	Graph orchestration
langchain-core	≥0.3.0	LangChain base
ollama	≥0.4.0	LLM client
duckdb	≥1.1.0	Analytical database
streamlit	≥1.41.0	Web UI
plotly	≥5.24.0	Visualizations
pandas	≥2.2.0	Data manipulation
python-dotenv	≥1.0.0	Environment variables
pydantic	≥2.0.0	Data validation
reportlab	any	PDF export

Table B.1: Python dependencies

Appendix C

Sample Queries and Expected Outputs

Query	Expected output	Chart
How many patients per province?	Table with provinces and counts	Bar
What is the average age?	Single number (e.g., 44.3 years)	None
Distribution of patient ages?	Age frequency table	Histogram
Patients with diabetes?	List of patient IDs	None
Diabetes and hypertension?	Count (e.g., 4 patients)	None
Medications for diabetic patients?	List of medication names	Table
Conditions in Granada?	List of condition names	Table
Procedures per year?	Year-count pairs	Area
Gender split in Almería?	Male/female percentages	Pie

Table C.1: Representative query test cases